AgriCare: Requirements, Design Model and Prototype

Group: The Equal SQLs • CSC312 Software Engineering • University of the Western Cape

Yanga Madosi
4354243
4354243@myuwc.ac.za

Tumelo Aphane (Group Leader)
4351723
4351723@myuwc.ac.za

Caleb Meyer
4359047
4359047@myuwc.ac.za

Nhlamulo Maluleka
4358074
4358074@myuwc.ac.za

Tanya Hadebe
4335167
4335167@myuwc.ac.za

ABSTRACT

AgriCare is a web-based system that helps small-scale and rural farmers, animal-care facilities and wildlife sanctuaries manage animal records, vaccinations, illness, quarantine, treatments, disease and weather alerts, and livestock security incidents. This report documents the analysis, design and first prototype of the system. The user model profiles six user roles and an external weather service and maps their tasks to eleven use cases; the mental model presents AgriCare as a familiar digital farm office with a sick bay; and the requirements analysis models the system's data, functions and behaviour. The design model then specifies a layered REST architecture, a relational data design, typed design classes, pseudocode for the safety-critical components and a user-interface design guided by the golden rules. Finally, an evolutionary program-code prototype built with FastAPI and SQLAlchemy implements a traced vertical slice covering animal files, illness triage, quarantine, vaccinations, outbreak flags and auditing, and its evaluation feeds back into the design.

1 INTRODUCTION

1.1 Project Scope and Context

AgriCare is a web-based livestock and animal-health management system for small-scale and rural farmers, animal-care facilities and wildlife sanctuaries. Many of these users still keep health information in paper registers, vaccination cards and message threads, so overdue vaccinations are missed, sick animals are isolated late, and the early signs of an outbreak are easy to overlook. AgriCare centralises animal records, vaccinations, illness reports, quarantine, treatments, disease-outbreak and severe-weather alerts, and livestock security incidents in one system [1]. It will be built with a React front end, a Python/FastAPI back end and a MySQL database, following an Agile/Scrum process of two-week sprints. This report presents the user model, mental model and requirements analysis (Sections 2–4), the design model (Section 5) and a working program-code prototype (Section 6), followed by the software engineering practices applied (Section 7).

1.2 User Needs

The main users are farmers and livestock owners, farm workers and animal-care staff (including authorised family members), veterinarians, facility and sanctuary managers, system administrators and auditors. Farmers need a simple way to register and monitor livestock, record health events, receive health and weather alerts, and identify animals that need attention. Farm workers need quick access to the animals in their care and the ability to update records during daily rounds. Veterinarians need complete histories and control over clinical decisions such as quarantine, treatment and discharge. Managers need an overview of health status across enclosures, while administrators manage accounts and permissions, and auditors need read-only access to trustworthy records.

1.3 Data Collection Methods

User needs were identified from the stakeholder analysis in our project proposal [1], a review of paper-based practice (herd registers, vaccination cards and treatment charts), an analysis of existing livestock-management tools, and publicly available biosecurity guidance such as the WOAH Terrestrial Animal Health Code [2]. Each group member also walked through a typical day for one user class (for example, a farm worker's morning round) to identify tasks, frequency, devices and environmental constraints such as gloves, bright sunlight and weak mobile signal. The existing tools the group reviewed focused on production (weights and breeding) rather than biosecurity, and none of them linked individual illness records to herd-level outbreak signals in the way proposed for AgriCare. These findings should be validated with at least one practising veterinarian and one farmer before the design is finalised.

2 USER MODEL

2.1 User Profiles

The user model profiles every end user of AgriCare and the tasks they must be able to perform [3]. A role is not necessarily a distinct person: on a small farm one individual may hold both the owner and manager roles, and the system grants the union of their permissions.

2.1.1 Care Staff and Farm Workers. Farm workers, stock handlers, sanctuary keepers and authorised family members are closest to the animals. Their digital literacy ranges from basic to moderate; most are confident with messaging apps but not with long forms. They use AgriCare mainly on a smartphone in pens and enclosures, often wearing gloves and with patchy signal. Their goals are to find an animal quickly, record symptoms and observations without slowing down their rounds, and know which tasks are due. They need tag-first search, short forms, large controls and confirmation that a report was received.

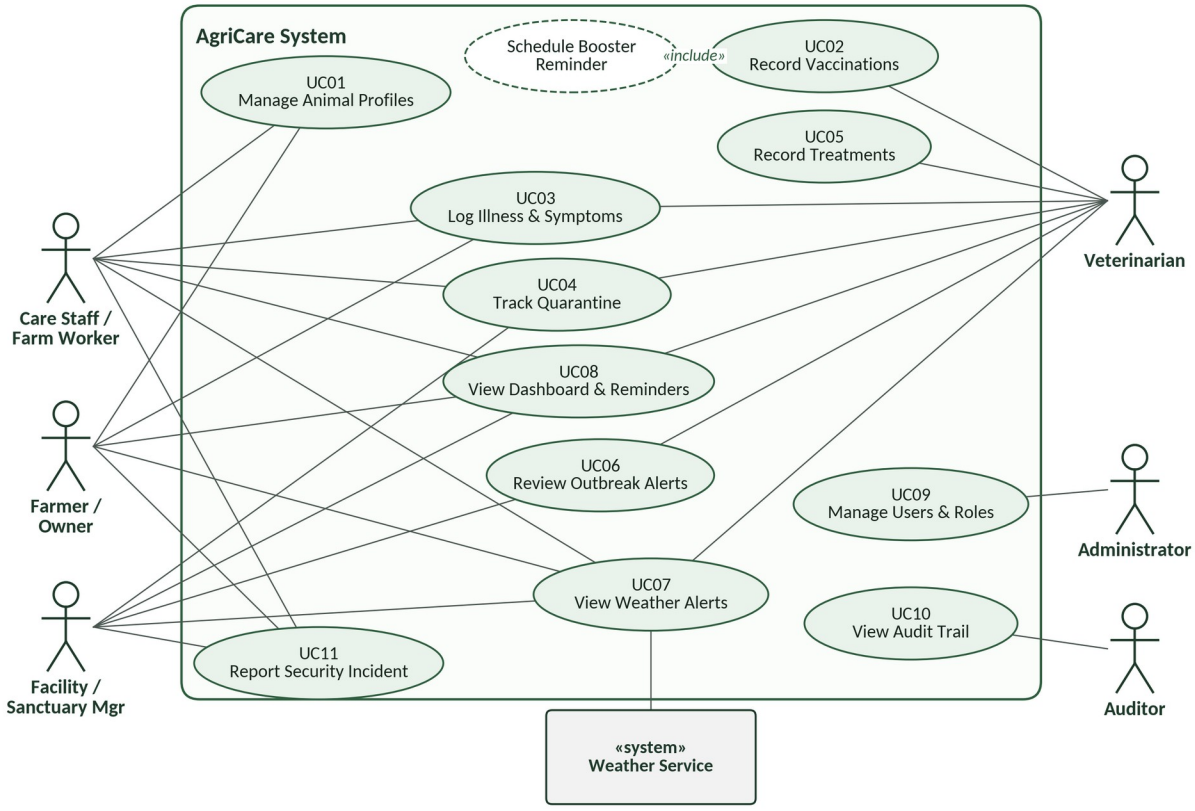


Figure 1: UML use-case diagram for AgriCare showing all end users and system actors.

2.1.2 Veterinarians. Veterinarians hold clinical authority and often serve several farms or facilities. They are comfortable with software and need complete histories (vaccination batches, treatments, previous illness), a severity-ordered list of cases and immediate notification of urgent reports. Their frustration with current practice is incomplete or verbal information. Only they may record vaccinations and treatments, move animals through quarantine and approve discharge.

2.1.3 Farmers and Livestock Owners. Owners range from spreadsheet users to people who still prefer a paper register. They use a phone in the field and occasionally a computer at home. They want to know what each animal has had, what is due and whether any alert affects their stock, and to record theft or missing animals. They need plain wording, status at a glance and the ability to update their own animals' records without making clinical decisions.

2.1.4 Facility and Sanctuary Managers. Managers oversee health and biosecurity procedures across a facility. They typically work from a desktop in the office and need enclosure-grouped overviews of overdue vaccinations, quarantined animals and illness clusters, and must follow up outbreak, weather and security alerts.

2.1.5 Administrators and Auditors. Administrators are technically skilled occasional users who create and deactivate accounts and assign roles; following least privilege, they do not need access to clinical records. Auditors, including animal-health officials acting in a compliance role, need read-only access to records and an immutable audit trail showing who changed what and when.

2.1.6 External Actors. The **Weather Service** is an external system that supplies forecasts for each farm location. An internal scheduler also triggers vaccination reminders and periodic

outbreak checks; it is described in the requirements rather than modelled as a separate actor.

2.2 Use-Case Model

Figure 1 maps each actor to the use cases it initiates [4, 5]. Every use case requires the actor to be authenticated (FR-01), so log-in is treated as a precondition rather than drawn separately. Recording a vaccination always schedules the next booster reminder, shown as an «include» relationship. Table 1 summarises the goal of each use case.

Table 1: Summary of AgriCare use cases

ID	Use case	Primary actor(s)	Goal
UC01	Manage Animal Profiles	Care Staff, Farmer/Owner	Create, view, update, search and archive animal profiles identified by RFID or ear-tag ID.
UC02	Record Vaccinations	Veterinarian	Record vaccine, batch, dosage and date; the next due date is calculated and a reminder scheduled.
UC03	Log Illness & Symptoms	Care Staff, Farmer/Owner, Vet	Record symptoms, onset, severity (Low / Moderate / Urgent) and photos; Urgent reports raise an alert.
UC04	Track Quarantine	Vet (Care Staff and Manager view and add observations)	Move an animal only along Healthy → Suspect → Isolated → Monitored → Cleared, with reasons and discharge approval.
UC05	Record Treatments	Veterinarian	Record medication, dosage, route, schedule, recovery progress and adverse effects.
UC06	Review Outbreak Alerts	Vet, Facility Manager	Review clusters of similar illness in an enclosure or area; the vet confirms or dismisses the alert.
UC07	View Weather Alerts	All operational roles; Weather Service	See hazardous-weather warnings (heat, frost, storms) with mitigation advice.
UC08	View Dashboard & Reminders	All operational roles	See role-specific tasks, due and overdue vaccinations, quarantined animals and open alerts.
UC09	Manage Users & Roles	Administrator	Create, update and deactivate accounts and assign roles.
UC10	View Audit Trail	Auditor	Inspect the read-only log of who changed which record and when.
UC11	Report Security Incident	Care Staff, Farmer/Owner, Manager	Record suspected theft or missing animals, with date, location and status, and track follow-up.

2.3 Key Use-Case Descriptions

The three most safety-critical use cases are described below as sequences of steps, from the user's goal to its fulfilment [4]. Their extensions become the edge cases of our test-driven development test suites.

UC03 Log Illness & Symptoms	
Actor	Care Staff / Farm Worker (Vet notified)
Pre-condition	User is logged in; the animal exists.
Main flow	1. Scan or search the tag and open the animal file. 2. Select Report Illness; tick symptoms by body system, set onset time, add notes and photos. 3. Submit; the system saves the report, classifies severity and writes an audit entry. 4. Moderate or Urgent reports set the animal to Suspect and appear in the vet's queue.
Extensions	Urgent severity immediately alerts the on-duty vet and manager. Missing fields are flagged without losing entered data. Every report is included in the outbreak check (UC06).
Target	Completed in 60 seconds or less (NFR-06).

UC04 Track Quarantine	
Actor	Veterinarian (Care Staff record observations)
Pre-condition	Animal is Suspect or has an open quarantine session.
Main flow	1. Open the quarantine board and select the animal. 2. System shows current state, isolation location, observations and only the permitted next state. 3. Vet records an observation, selects the next state and gives a reason. 4. System validates the transition and role, stores it with time and user, and updates the board.
Extensions	Moving to Cleared requires the minimum monitoring period, no open Urgent report and recorded decontamination. Invalid transitions (e.g., Isolated → Cleared) are rejected with an explanation. Death or culling archives the record.

UC11 Report Security Incident	
Actor	Farmer/Owner, Care Staff or Manager
Main flow	1. Select Report Incident and choose the affected animal(s). 2. Enter incident type (theft, missing, injury by predator), date, location and notes. 3. System saves the incident as Open, notifies the manager and owner, and writes an audit entry. 4. Status is updated to Investigating, Resolved or Closed as follow-up happens.
Extensions	An animal reported missing is flagged on its profile until the incident is resolved.

2.4 Roles and Access Rights

Table 2 applies the least-privilege principle from the proposal [1]: each role holds only the "keys" it needs, clinical decisions are reserved for veterinarians, and auditors can read but never write.

Table 2: Role-based access (✓ full, R read-only, – none). CS = Care Staff, FO = Farmer/Owner, FM = Facility Manager, Adm = Administrator, Aud = Auditor

Capability	CS	FO	FM	Vet	Adm	Aud
Create / update animal profiles	✓	✓	✓	R	–	R
Record vaccinations and treatments	R	R	R	✓	–	R
Log illness and observations	✓	✓	R	✓	–	R
Change quarantine state / discharge	R	R	R	✓	–	R
Confirm or dismiss outbreak alerts	R	R	R	✓	–	R
Report security incidents	✓	✓	✓	R	–	R
View dashboard and alerts	✓	✓	✓	✓	–	R

Capability	CS	FO	FM	Vet	Adm	Aud
Manage users and roles	–	–	–	–	✓	–
View audit trail	–	–	R	–	✓	✓

3 MENTAL MODEL

3.1 Central Metaphor and Familiar Environment

A mental model is the picture a user carries of what a system contains and how it will respond; users build this picture from experience with familiar objects and use it to predict what an action will do [6]. The assignment brief illustrates this with an online shop that borrows the picture of a physical shop with a trolley and aisles [7]. Norman distinguishes three things that should agree: the designer's conceptual model, the user's mental model, and the system image — everything the user can actually see and read [6]; Cooper et al. warn that an interface exposing its implementation model instead forces users to learn the software's internals rather than their own task [8].

Paper-based animal care already suggests a natural picture: a filing cabinet holding one folder per animal, a vaccination card, a board showing which animals are unwell, an isolation pen reached through a gate, and keys held by different staff members. AgriCare is therefore presented as one consistent environment built from these familiar parts — a **digital farm office with a sick bay**. A single, coherent picture is easier to learn than several unrelated ones, and it follows the usability heuristic of matching the system to the real world rather than to internal system terms [9]. Four anchors carry this picture, each pairing a physical object with its AgriCare counterpart and the use cases it supports (Figure 2).

The Animal File (UC01, UC02, UC03, UC05). Each animal has one folder, labelled on its tab with the RFID or ear-tag ID (FR-02). Inside sit the front page (species, breed, sex, age, enclosure, photograph, weight, temperature), a vaccination card, illness notes and a treatment chart. Users open a file, read it, add to it and close it; nothing is discarded — retiring an animal moves its file to a closed drawer, matching the "archive" wording of FR-03 rather than deletion.

The Sick Bay and Gates (UC04). An animal that may be ill moves along a one-way path through a sequence of gates, rather than having a status flag switched on and off. Each gate represents a permitted quarantine-state transition, and passing through one requires an authorised role and is written to the logbook.

The Notice Board and Alarm Bell (UC06, UC07, UC08). The dashboard is the wall board a person checks at the start of a shift, filtered to show only what concerns their own role. Alerts are the bell: they draw attention to a possible issue, but a person — not the system — decides what to do about it.

The Key Ring and Logbook (UC09, UC10). Each role holds only the keys needed for the tasks it may perform, following the least-privilege principle stated in the proposal [1]. Every important change made through any of the other three anchors is written into a shared logbook that an auditor may read but never edit.

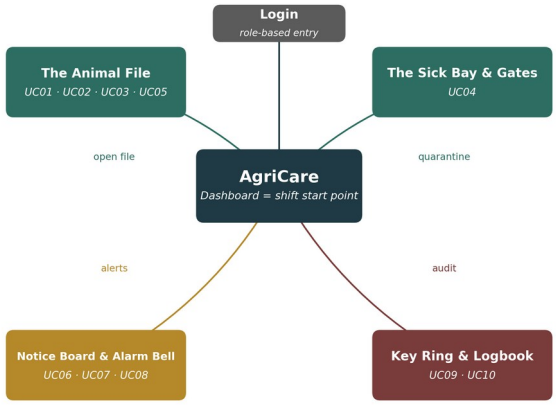


Figure 2: The four anchors of the AgriCare mental model around the central dashboard, with the use cases each anchor carries.

3.2 Mental Model by Role and Core Workflows

Each role carries its own working picture of the same four anchors, built around the tasks it performs most often, and the interface must surface the anchor relevant to that role first.

Managing animal profiles (UC01, FR-02–FR-04). Creating a profile is opening a new folder, and the ear-tag or RFID ID is the label on its tab; users look animals up by tag first, with species, name or enclosure as fallback fields. Archiving is filing away, not shredding — archived profiles remain searchable, so the interface offers "Archive" and no "Delete" control.

Recording vaccinations (UC02, FR-05–FR-06). A vaccination entry is a new line on the animal's vaccination card — vaccine, batch or lot number, manufacturer, dosage, date and administering veterinarian. What the paper card cannot do, and AgriCare adds, is the booster reminder, read by the user as a due date pencilled into the margin; an overdue vaccination is simply that date having passed.

Logging illnesses (UC03, FR-07). A report is a dated note added to the file. Its severity level (Low, Moderate, Urgent) follows the logic of triage; because colour alone is unreliable for users with colour-vision deficiencies, each level is paired with a word and an icon [10]. An Urgent report rings the alarm bell and the

interface confirms that the alert was raised, while the report itself neither diagnoses nor quarantines an animal automatically.

Table 3: Working picture, expected workflow and interface consequence for each role

Role (use cases)	Working picture	Expected workflow	Interface consequence
Care Staff / Farm Worker (UC01, 03, 04, 07, 08, 11)	The daily round with a clipboard: walk the enclosures, note what is wrong, move on.	Find the animal by tag, record signs and save; check quarantined animals and reminders.	Tag-first search; short forms usable on a phone; 44×44 px controls; a failed save keeps other entries. No prescribing or discharge approval.
Veterinarian (UC02–UC08)	The consulting room: open the file, read the history, write the entry.	Review histories; record vaccines and treatments; judge severity, clearance and outbreak flags.	Whole history readable in one place. Clinical decisions are entered by the veterinarian, never inferred by the system.
Farmer / Owner (UC01, 03, 07, 08, 11)	The stock register: what has each animal had, and what is due?	Look up animals, vaccination status and alerts; update own animals; report theft or missing animals.	Plain wording and status at a glance; may edit own animals' profiles and report incidents, but makes no clinical decisions.
Facility / Sanctuary Manager (UC04, 06, 07, 08, 11)	The notice board: the whole facility, grouped by enclosure.	Scan overdue doses, quarantine and illness by enclosure; open alerts and incidents and follow up.	Enclosure-grouped overview; the same role serves both farms and sanctuaries.
Administrator (UC09)	The key-holder: issues and withdraws keys.	Create and disable accounts; assign roles.	Access settings kept apart from clinical screens; no implied approval authority.
Auditor (UC10)	The inspector with the logbook: reads, never writes.	Inspect audit events and record histories.	Read-only views showing who changed what, and when.
Weather Service (UC07; external)	The radio bulletin: arrives from outside and can go stale.	Supplies hazard warnings that users then view.	Show the time of the data and state clearly when it could not be refreshed (NFR-03).

Tracking quarantine (UC04, FR-08). Users see an animal's present state, its isolation location and its observation history, and may request only the next permitted state along the Healthy → Suspect → Isolated → Monitored → Cleared path, reflecting one-way movement through the sick bay's gates rather than a free choice among all five states.

Viewing dashboards and alerts (UC06–UC08, FR-10–FR-12). Each role's notice board opens on the part of the farm office that concerns its job — a care worker's board starts from reminders and quarantined animals, a manager's from overdue doses and enclosures with illness, and a veterinarian's from urgent cases — while outbreak alerts behave as a smoke detector rather than a confirmed diagnosis, prompting review rather than declaring an outcome. Table 3 summarises the working picture, expected workflow and resulting interface requirement for every role in the user model.

4 REQUIREMENTS ANALYSIS

4.1 Approach

Requirements were analysed from three views [11, 12]: **data** (what AgriCare must store, modelled as a high-level class diagram), **function** (what it must do, captured in the use cases and functional requirements) and **behaviour** (how records change over time, modelled as a state machine). Each requirement was prioritised as *must-have* (needed for a working first release) or *nice-to-have* (desirable if time allows), and traced to the use cases in Section 2.

4.2 Data Requirements

Table 4 lists the categories of data AgriCare must store and the class that holds each. Figure 3 models these classes at a high (abstract) level; attribute types, keys and full operations are added in the design model (Figure 7). Composition (filled diamond) marks records that cannot exist without their owner, such as a vaccination record without its animal, and the three alert types specialise an abstract Alert class.

Table 4: Data requirements and the classes that store them

Data category	Stored data	Class
User	User ID, name, contact details, role, secure credentials	User
Animal	Tag ID, species, breed, sex, age, owner/facility, enclosure, health status	Animal, Enclosure, Facility
Vaccination	Vaccine, batch, dosage, date given, next due date, responsible vet	VaccinationRecord
Treatment	Medication, dosage, route, dates, reason, vet, notes, adverse effects	Treatment
Health observation	Symptoms, severity, onset, photos, recorded by	IllnessReport
Quarantine	State, reason, isolation location, observations, discharge	QuarantineSession
Alerts	Type, message, date, severity, affected animal or area	Alert and subclasses
Weather	Hazard type, location, severity, start and end time	WeatherAlert
Disease / outbreak	Suspected condition, affected area and animals, alert level	OutbreakAlert
Security incident	Animal, incident type, date, location, status, notes	SecurityIncident
Audit	User, action, record affected, old and new values, date/time	AuditLog

4.3 Functional Requirements

Table 5 combines the functional requirements of the proposal with those identified during this analysis. FR-01 to FR-14 are must-have features; FR-15 to FR-18 are nice-to-have features.

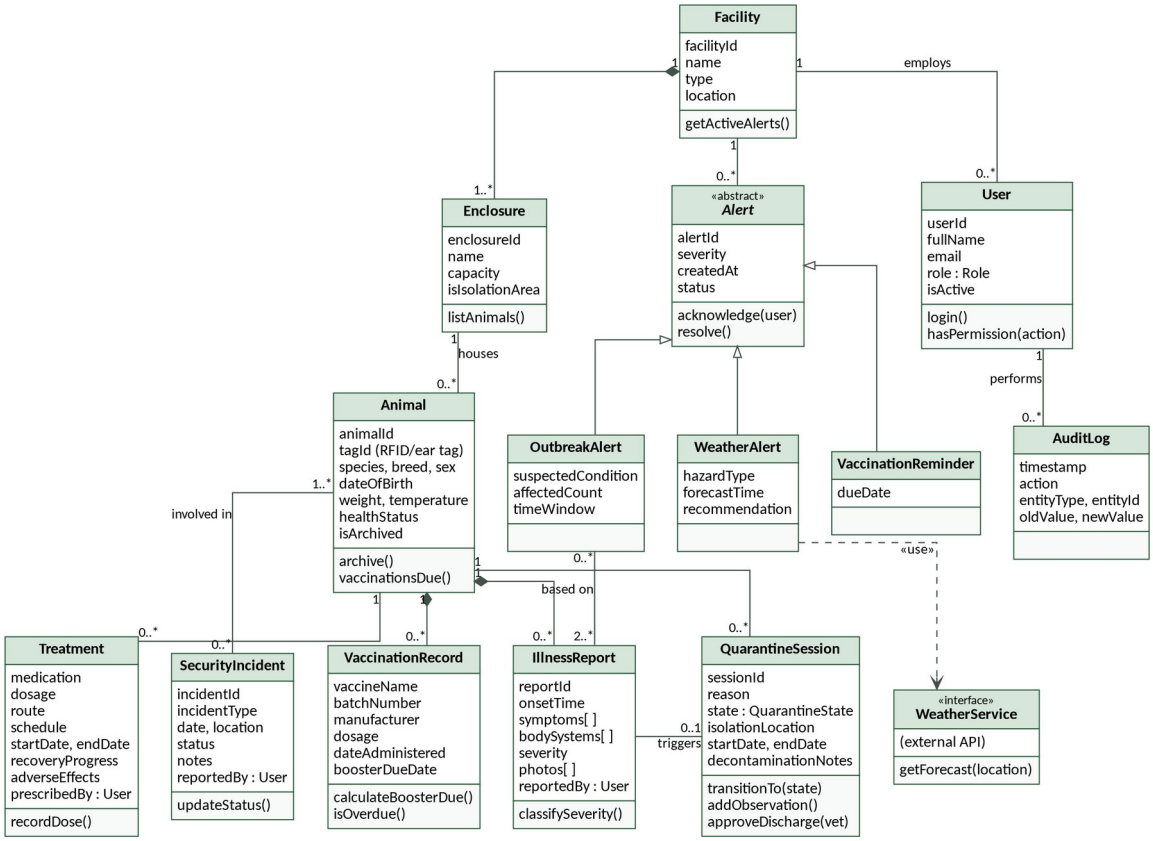
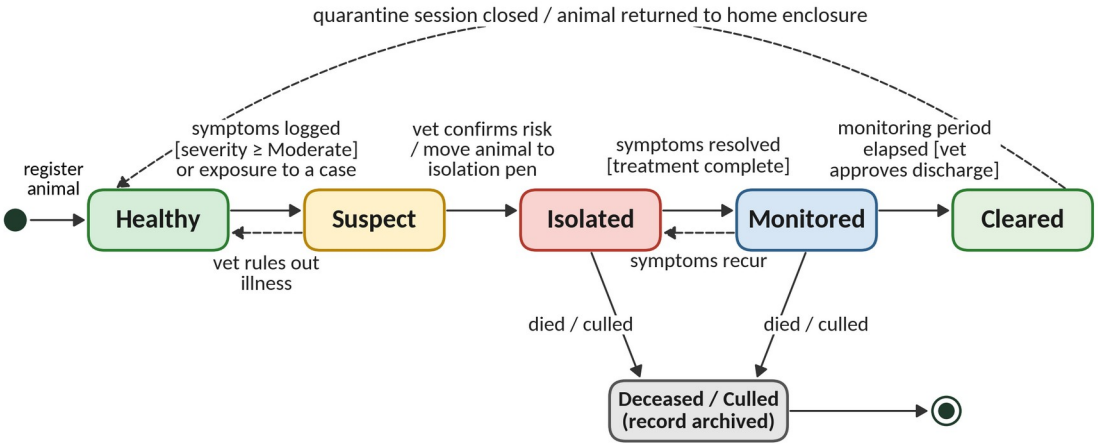


Figure 3: High-level UML class diagram of the AgriCare domain.

Table 5: Functional requirements

ID	Description	Priority	UC
FR-01	Users must register, log in securely and manage their accounts; access is granted by role (Care Staff, Farmer/Owner, Manager, Vet, Administrator, Auditor).	Must	UC09
FR-02	Authorised users must register animal profiles with tag ID, species, breed, sex, age, enclosure, photo, weight and temperature.	Must	UC01
FR-03	Authorised users must update profiles and archive, never delete, retired animals.	Must	UC01
FR-04	Users must search for and view animal profiles by tag ID or other identifying details.	Must	UC01
FR-05	Vets must record vaccinations (vaccine, batch, manufacturer, dosage, date); the system calculates the next due date.	Must	UC02
FR-06	The system must retain vaccination history and flag upcoming and overdue vaccinations with reminders.	Must	UC02, UC08
FR-07	Authorised users must log illnesses and health observations; reports are classed Low, Moderate or Urgent, and Urgent reports raise an alert.	Must	UC03
FR-08	The system must manage quarantine through Healthy → Suspect → Isolated → Monitored → Cleared, recording reasons, location, observations and vet discharge approval.	Must	UC04
FR-09	Vets must record treatments and medication; users can view treatment and observation history.	Must	UC05
FR-10	The system must detect possible outbreaks from similar symptoms, shared enclosures and time periods, and display disease alerts.	Must	UC06
FR-11	The system must provide severe-weather alerts with mitigation advice using a weather API.	Must	UC07
FR-12	The system must provide role-specific dashboards summarising health, vaccination, quarantine and alert information.	Must	UC08
FR-13	Authorised users must record livestock theft or security incidents and track their status.	Must	UC11
FR-14	The system must keep an audit trail of important changes that auditors can read but not edit.	Must	UC10
FR-15	Users should be able to search and filter livestock and health records.	Nice	UC01
FR-16	Users should receive notifications for important health, weather and security events.	Nice	UC08
FR-17	AgriCare could provide English and isiZulu language support.	Nice	All
FR-18	The system could show basic statistics and trends from livestock records.	Nice	UC08



Note: no direct transition exists from Healthy to Cleared, or from Suspect/Isolated to Cleared. Only a Veterinarian may fire the guarded transitions.

Figure 4: UML state machine for an animal's quarantine status.

4.4 Non-Functional Requirements

The non-functional requirements in Table 6 describe how well AgriCare must operate. Where possible they carry the measurable targets set in the proposal [1], so that each can be verified by testing.

Table 6: Non-functional requirements

ID (Category)	Description	Priority
NFR-01 (Security)	Sensitive user and livestock data is accessible only to authorised users; passwords are hashed, traffic uses HTTPS, and personal data is handled in line with POPIA [13].	Must
NFR-02 (Security)	Role-based access is enforced on the server for every function (least privilege).	Must
NFR-03 (Reliability)	Stored records remain available (target 99.5% availability); if the weather service fails, the last forecast is shown with its time.	Must
NFR-04 (Integrity)	Incomplete or invalid records cannot be saved; records are archived rather than deleted.	Must
NFR-05 (Performance)	Common actions such as opening a profile respond in under 300 ms for 95% of requests with 100 concurrent users.	Must
NFR-06 (Usability)	The interface suits users with different digital experience; an illness report takes 60 s or less, and errors give clear instructions.	Must
NFR-07 (Accessibility)	Clear labels and simple navigation; touch targets of at least 44×44 px; status never shown by colour alone [10].	Must
NFR-08 (Compatibility)	Works on modern browsers on phones, tablets and desktops.	Must
NFR-09 (Maintainability)	Modular structure so animals, health records, alerts, security and users can be maintained independently.	Nice

4.5 Behavioural Model

Quarantine is the most rule-heavy behaviour in AgriCare, and an error here has real consequences: an infectious animal returned to the herd too early can spread disease. Figure 4 models an animal's quarantine status as a UML state machine. Guards appear in square brackets, dashed transitions are "backward" paths (a suspicion ruled out or symptoms recurring), and any transition not shown is rejected by the system.

The remaining behaviour is captured as business rules that will be unit-tested before implementation. The next booster date is the date administered plus the vaccine's booster interval, and a vaccination is overdue once that date passes without a newer dose. A report is Urgent if it includes a red-flag sign (breathing difficulty, collapse, mouth or hoof lesions, or sudden death in the enclosure), Moderate if it includes fever or two or more symptoms, and otherwise Low. A possible outbreak is flagged when three or more animals in one enclosure report symptoms in the same body system within 72 hours. Weather alerts are raised for temperatures of 35 °C or more, frost, winds of 60 km/h or more, or an official severe-weather warning. All thresholds are configurable defaults that must be confirmed by a veterinarian.

5 DESIGN MODEL

The design model turns the analysis models of Sections 2–4 into a blueprint for construction. Following Pressman and Maxim [12, Ch. 12–15], it is organised into the architectural, data, class and component-level design elements, followed by the user-interface design. Each element traces back to the requirements and forward to the Step 5 prototype (Section 6), which already implements a vertical slice of this design. Where the prototype is deliberately simpler than the target design, the difference is stated.

5.1 Design Concepts Applied

Table 7 shows how the fundamental design concepts are applied in AgriCare. They explain why the code is organised as it is and give reviewers a checklist for later sprints.

Table 7: Design concepts and how AgriCare applies them

Concept	Application in AgriCare
Abstraction	*Procedural*: `classify_severity()` hides the triage rules behind one call. *Data*: an Animal object hides how its history is stored.
Architecture	Layered client–server style with a REST interface (Section 5.2).
Patterns	Layers; Unit of Work and Data Mapper through the SQLAlchemy session and models [15]; Adapter for the weather provider; a state-transition table for quarantine.
Separation of concerns, modularity	Backend split into `database`, `models`, `schemas`, `rules` and `main` modules; browser assets kept separately.
Information hiding	Clients see only REST contracts and validated schemas, so tables and rule internals can change without breaking the user interface.
Functional independence	Functions in `rules.py` are pure (no database or HTTP access): high cohesion, low coupling, unit-testable in isolation.
Refinement	Step 4 analysis classes (Figure 3) are refined into typed design classes (Figure 7) and then into code.
Refactoring	Planned: split `main.py` into one router per feature (animals, clinical, audit) as the API grows.

5.2 Architectural Design

AgriCare uses a **layered architecture** deployed as a three-tier client–server system (Figure 5). Each layer calls only the layer below it. The presentation layer runs in the browser and talks to the API layer over HTTP with JSON. The API layer, built with FastAPI [16], validates every request against a schema and checks the caller's role before any rule runs. The domain layer holds the business rules from Section 4.5, and the data-access layer uses the SQLAlchemy ORM [17], so the same code can run on SQLite for the prototype or MySQL in production by changing one setting (`DATABASE_URL`).

Putting the rules in their own layer means every client is protected by the same checks: a React web client or a future mobile app cannot skip the quarantine rules, because those rules live on the server. Role checks, audit logging and error codes are cross-cutting concerns applied in the API and domain layers. External services (the weather provider and an SMS or e-mail gateway) are reached through adapter classes with a cache, so a failed external call cannot stop core record-keeping (NFR-03).

Three architectural styles were compared before this one was chosen (Table 8). The layered REST style was preferred because it gives clear boundaries and testable rules while remaining a single deployable unit that a five-person student team can operate.

Table 8: Architectural alternatives considered

Style	Strengths	Weaknesses for AgriCare
Server-rendered MVC monolith	Simplest to build and host	Ties the interface to the server; harder to add the planned React and mobile clients
Microservices	Independent scaling and deployment	Too much operational overhead for one team and one facility-scale database
Layered REST (chosen)	Clear layers, rules testable in isolation, one deployable unit	Needs discipline to keep rules out of endpoints; addressed by `rules.py`

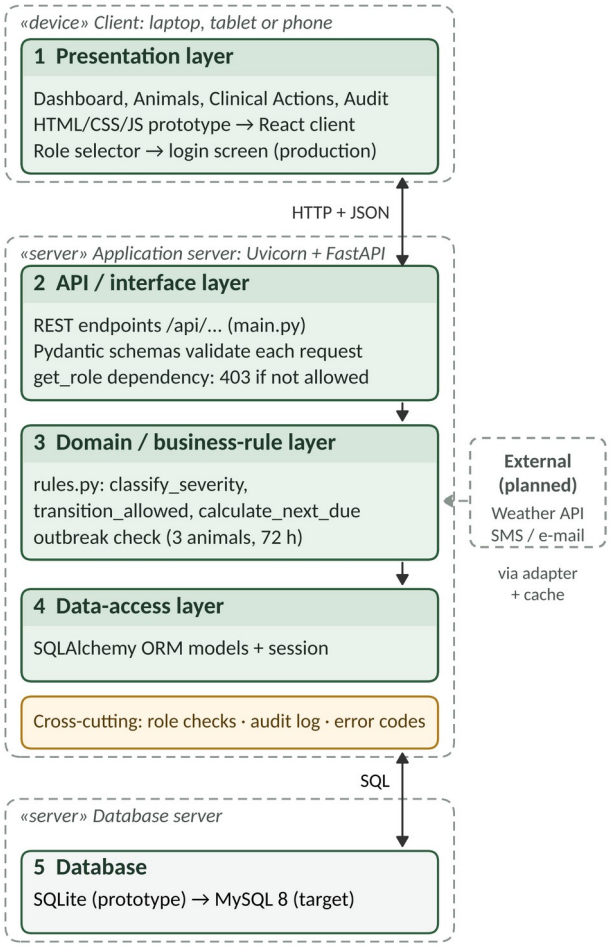


Figure 5: Layered architecture and deployment nodes. Solid boxes exist in the prototype; dashed external services are planned.

The API is the contract between the client and server components. Table 9 lists the endpoints that exist in the prototype, with the roles the server accepts for each. Any other role receives HTTP 403, and an unknown role receives HTTP 400.

Table 9: REST interface of the prototype (all paths start with /api)

Endpoint	Roles accepted	Purpose (traces to)
GET /animals	All	List active animal files (UC01)

Endpoint	Roles accepted	Purpose (traces to)
POST /animals	Care Staff, Farmer, Manager	Create a profile; 409 if tag exists (FR-02)
GET /animals/{id}/history	All	Animal file with illness, vaccination and quarantine history (FR-04)
POST /animals/{id}/illness	Care Staff, Farmer, Vet	Report illness; returns severity (UC03, FR-07)
POST /animals/{id}/vaccinations	Vet	Record dose; returns next due date (UC02, FR-05)
POST /animals/{id}/quarantine	Vet	Request a state change; 400 if not allowed (UC04, FR-08)
GET /dashboard	All	Dashboard counts; the same for every role in the prototype (UC08, FR-12)
GET /outbreak-alerts	All	Open and closed outbreak flags (UC06, FR-10)
GET /audit	Manager, Admin, Auditor	Latest 100 audit entries (UC10, FR-14)

Deployment. In the prototype all three nodes run on one laptop, with SQLite as a single file. In production the application server (Uvicorn behind an HTTPS reverse proxy) and a MySQL server run on a cloud or university virtual machine, and the client is any modern browser on a laptop, tablet or phone (NFR-01, NFR-08). Planned endpoints for the next sprint are login, treatments, alert acknowledgement and security incidents.

5.3 Data Design

Figure 6 (on the following landscape page) maps the analysis classes onto a relational schema for MySQL using crow's-foot entity–relationship notation [18]. Tables with a dark header already exist in the prototype, although in simplified form. The main design decisions are as follows.

- **Append-only history.** Every clinical event (vaccination, illness report, quarantine event, treatment) is a separate row linked to `animal`, so history is never overwritten (BR-09). The current quarantine state is also stored on `animal` for fast dashboard queries. This deliberate denormalisation stays consistent because only the quarantine service changes it.

- **Single-table inheritance for alerts.** The Alert class hierarchy maps to one `alert` table with an `alert_type` discriminator, because the subclasses share most columns and the dashboard reads all alert types together. The prototype's `outbreak_alerts` table becomes `alert` rows of type Outbreak.

- **Quarantine session and event.** A `quarantine_session` records one isolation episode (reason, isolation pen and discharge), and each `quarantine_event` records one state change. This settles the naming difference between the Step 4 model (QuarantineSession) and the prototype (QuarantineEvent): the design keeps both, and the prototype implements the events while leaving the session implicit.

- **Integrity constraints.** Unique `tag_id` and `email`; NOT NULL on required fields; ENUM values for states, severity and roles; and foreign keys with ON DELETE RESTRICT, so no health record can be orphaned or deleted (FR-03, NFR-04). Indexes on every `animal_id` and on (`enclosure_id`, `body_system`, `created_at`) support the 72-hour outbreak query.

- **Privacy.** Personal data is limited to what each role needs, and passwords are stored only as bcrypt hashes (NFR-01, POPIA [13]).

In the prototype, the enclosure is a text column on `animals` and the acting role is stored as text rather than as a user foreign key. Both become proper foreign keys when login and the `enclosure` table are added.

5.4 Class Design

Figure 7 (landscape) refines the Step 4 analysis class diagram (Figure 3) into design classes with visibility, attribute types and operation signatures. Attributes are private (–) and are changed only through public operations (+) that enforce the business rules. For example, `Animal.state` changes only through `QuarantineSession.transitionTo()`, which validates the transition and returns the QuarantineEvent it creates. Enumerations replace free text for roles, quarantine states and severity, and `SymptomCode` replaces free-text symptoms (Section 5.6.4). References to User are shown as typed attributes rather than lines so that the diagram stays readable.

The design classes fall into Pressman's five categories [12]: *user-interface classes* (the browser views), *business-domain classes* (Figure 7), *process classes* (the rule functions, the outbreak detector and the audit service), *persistence classes* (the SQLAlchemy models and session) and *system classes* (configuration through `DATABASE_URL` and the role dependency). In code, the domain classes are the SQLAlchemy models in `models.py`, and the Pydantic schemas in `schemas.py` act as data-transfer objects at the API boundary.

5.5 Component-Level Design

Table 10 lists the components, their responsibilities and the interfaces they provide. Listings 1–3 give the procedural design of the three most safety-critical components in pseudocode. Lines marked *design* are specified here but not yet implemented in the prototype.

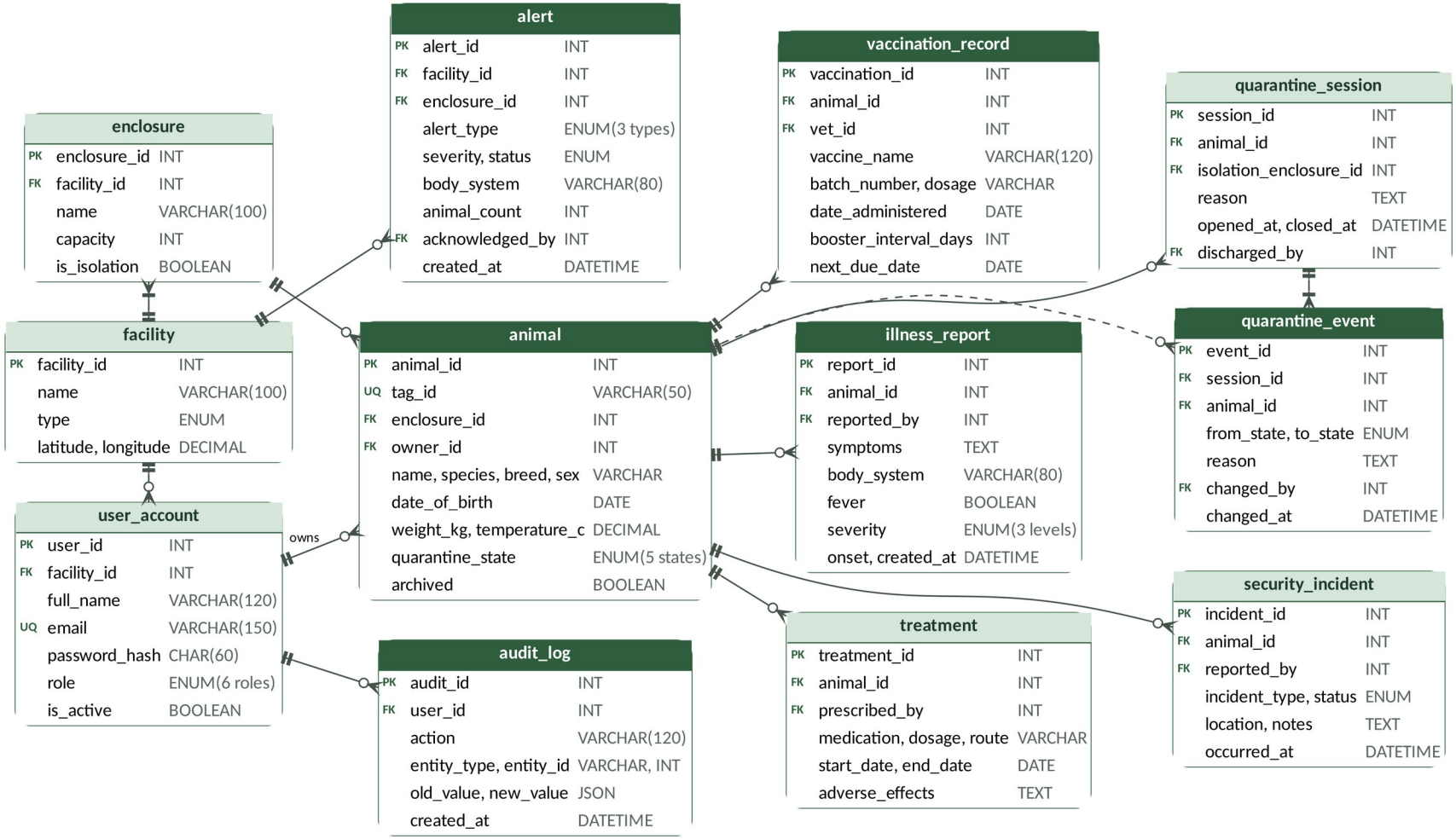


Figure 6: Entity–relationship diagram of the target MySQL schema (crow's-foot notation: || one, o< zero or many, |< one or many). Dark headers mark tables already implemented in the prototype; light headers are added in later sprints.

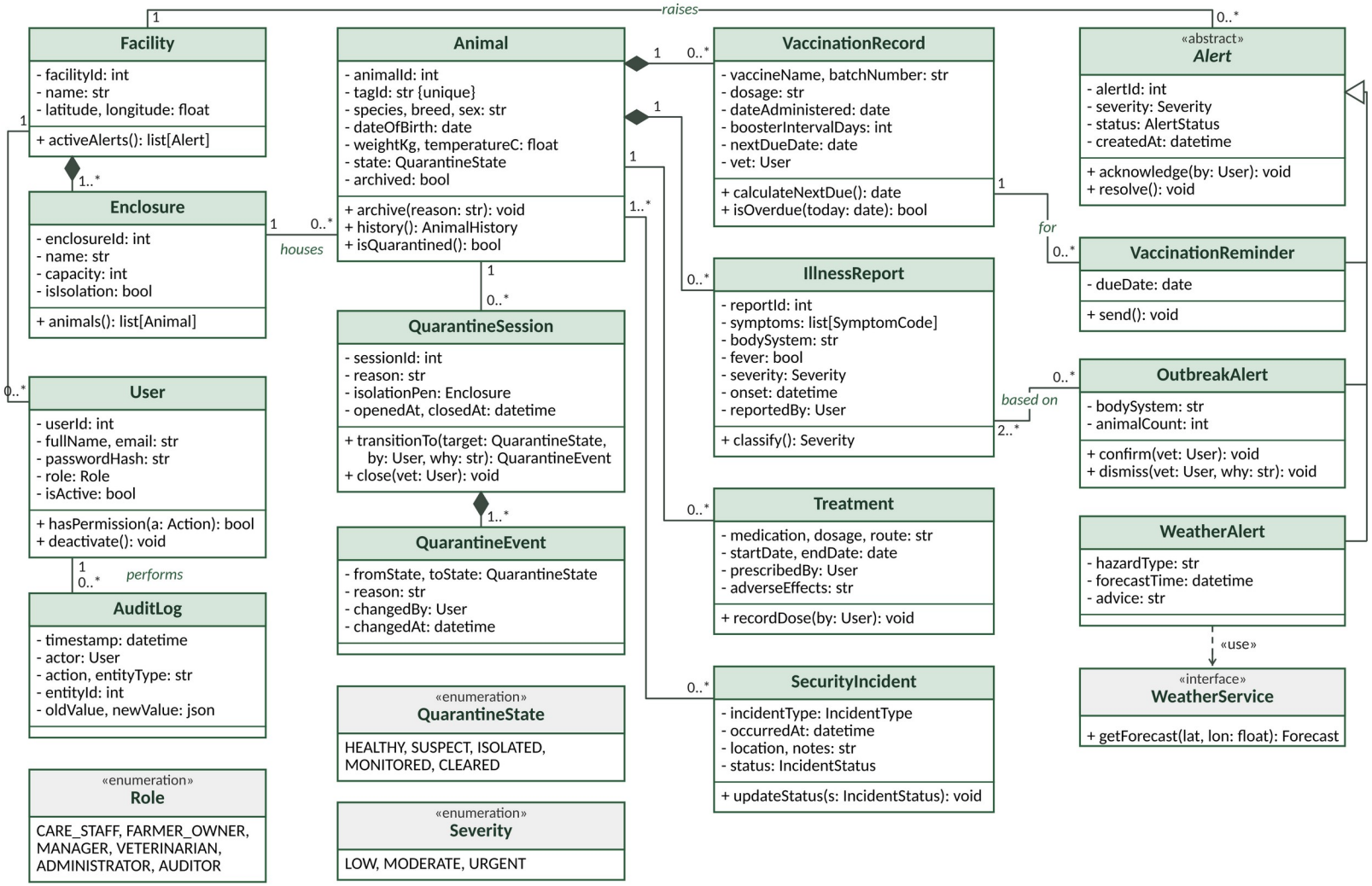


Figure 7: Design class diagram refining Figure 3 with visibility, attribute types and operation signatures. References to User appear as typed attributes to keep the diagram readable.

Table 10: Components, responsibilities and interfaces

Component	Responsibility	Interface / code
AnimalRegistry	Create, search and archive animal files; unique tag	/animals; `Animal`
IllnessTriage	Store report, classify severity, flag Suspect, trigger outbreak check	/illness; `classify_severity`
QuarantineController	Check role and transition; write event	/quarantine; `transition_allowed`
VaccinationScheduler	Record dose, compute due date, flag overdue doses	/vaccinations; `calculate_next_due`
OutbreakDetector	Count distinct sick animals per enclosure and body system in 72 h	`update_outbreak_alert`
AuditService	Append audit entries; read-only access for audit roles	/audit; `audit()`
WeatherAdapter (planned)	Fetch and cache forecasts; raise weather alerts	`WeatherService` interface

Listing 1: Illness reporting and severity classification (UC03)

```
PROCEDURE ReportIllness(animalId, symptoms,
                        bodySystem, fever, user)
  IF user.role NOT IN {CARE_STAFF, FARMER, VET}
     THEN RETURN 403
  animal <- Animals.find(animalId)
  IF animal = NULL OR animal.archived THEN RETURN 404
  severity <- ClassifySeverity(symptoms, fever)
  SAVE IllnessReport(animal, symptoms, bodySystem,
                     severity, user)
  IF severity IN {MODERATE, URGENT}
     AND animal.state = HEALTHY THEN
     animal.state <- SUSPECT
     SAVE QuarantineEvent(HEALTHY, SUSPECT, SYSTEM)
  CheckOutbreak(animal.enclosure, bodySystem)
  IF severity = URGENT THEN NotifyVet()  // design
  Audit("REPORT_ILLNESS", user); COMMIT
  RETURN 201 (report with severity)

FUNCTION ClassifySeverity(symptoms, fever)
  IF symptoms ∩ RED_FLAGS ≠ ∅ THEN RETURN URGENT
  IF fever OR |symptoms| ≥ 2 THEN RETURN MODERATE
  RETURN LOW

PROCEDURE CheckOutbreak(enclosure, bodySystem)
  n <- COUNT DISTINCT animal_id OF illness reports
       IN enclosure WITH bodySystem
       CREATED IN last 72 hours
  IF n ≥ 3 AND no open alert for (enclosure,
     bodySystem) THEN SAVE OutbreakAlert(n)
```

Listing 2: Quarantine state change (UC04)

```
ALLOWED <- { HEALTHY:   {SUSPECT},
             SUSPECT:   {HEALTHY, ISOLATED},
             ISOLATED:  {MONITORED},
             MONITORED: {ISOLATED, CLEARED},
             CLEARED:   {} }

PROCEDURE ChangeState(animalId, target, reason, user)
  IF user.role ≠ VET THEN RETURN 403
  IF length(reason) < 3 THEN RETURN 422
  animal <- Animals.find(animalId)
  IF animal = NULL THEN RETURN 404
  current <- animal.state
  IF target NOT IN ALLOWED[current] THEN
     RETURN 400 "Invalid transition"
  IF target = CLEARED AND NOT DischargeOK(animal)
     THEN RETURN 400   // design: BR-06
  SAVE QuarantineEvent(current, target, reason, user)
  animal.state <- target
  Audit("CHANGE_QUARANTINE", user); COMMIT
  RETURN 201 (event)
```

Listing 3: Vaccination due date and overdue check (UC02, FR-06)

```
PROCEDURE RecordVaccination(animalId, data, user)
  IF user.role ≠ VET THEN RETURN 403
  REQUIRE 0 < data.intervalDays ≤ 3650
  nextDue <- data.dateGiven + data.intervalDays
  SAVE VaccinationRecord(animalId, data, nextDue)
  Audit("RECORD_VACCINATION", user); COMMIT

FUNCTION IsOverdue(record, today)
  RETURN today > record.nextDue
     AND NOT EXISTS a newer dose of the same
         vaccine for the same animal  // design
```

5.6 User Interface Design

Interface design followed the analysis, design, construction and validation cycle of Pressman's Chapter 15 [12]. The four subsections below cover each part of that cycle.

5.6.1 Interface Analysis. The *users* and their digital literacy are profiled in Section 2.1, and their *tasks* are the use cases in Section 2.2. The *content* is the animal file, the alerts and the audit log. The *environment* is outdoors and in pens, often with gloves, bright sunlight, weak signal and a shared phone. These findings drive the large controls, short forms and tag-first search.

5.6.2 The Golden Rules. Table 11 applies Mandel's three golden rules [19] to AgriCare, each with a concrete design decision.

Table 11: Golden rules of interface design applied

Golden rule	Design decisions in AgriCare
Place the user in control	Every clinical change needs an explicit submit and a reason; errors explain how to recover in plain language; navigation is always visible, so users can leave any task.
Reduce the user's memory load	Tag-first lookup; defaults for today's date and vaccine; booster dates are calculated, not remembered; the dashboard shows what is due and who needs attention.
Make the interface consistent	The same layout on every screen (sidebar, page title, role box); the same status words and colours everywhere; the same toast confirmation for every action; use-case badges on forms.

5.6.3 Interface Design Steps. Interface objects and actions were taken from the use cases: an *animal file* is opened, created or archived; an *illness report* is submitted; a *vaccination* is added to the card; a *quarantine state* is requested; an *alert* is acknowledged, confirmed or dismissed; and the *audit log* is viewed. These became the screens and navigation in Figure 8, which follows the farm-office mental model: users start at the notice board (dashboard), open an animal's file, then act on it. Figure 9 shows the screens built so far.

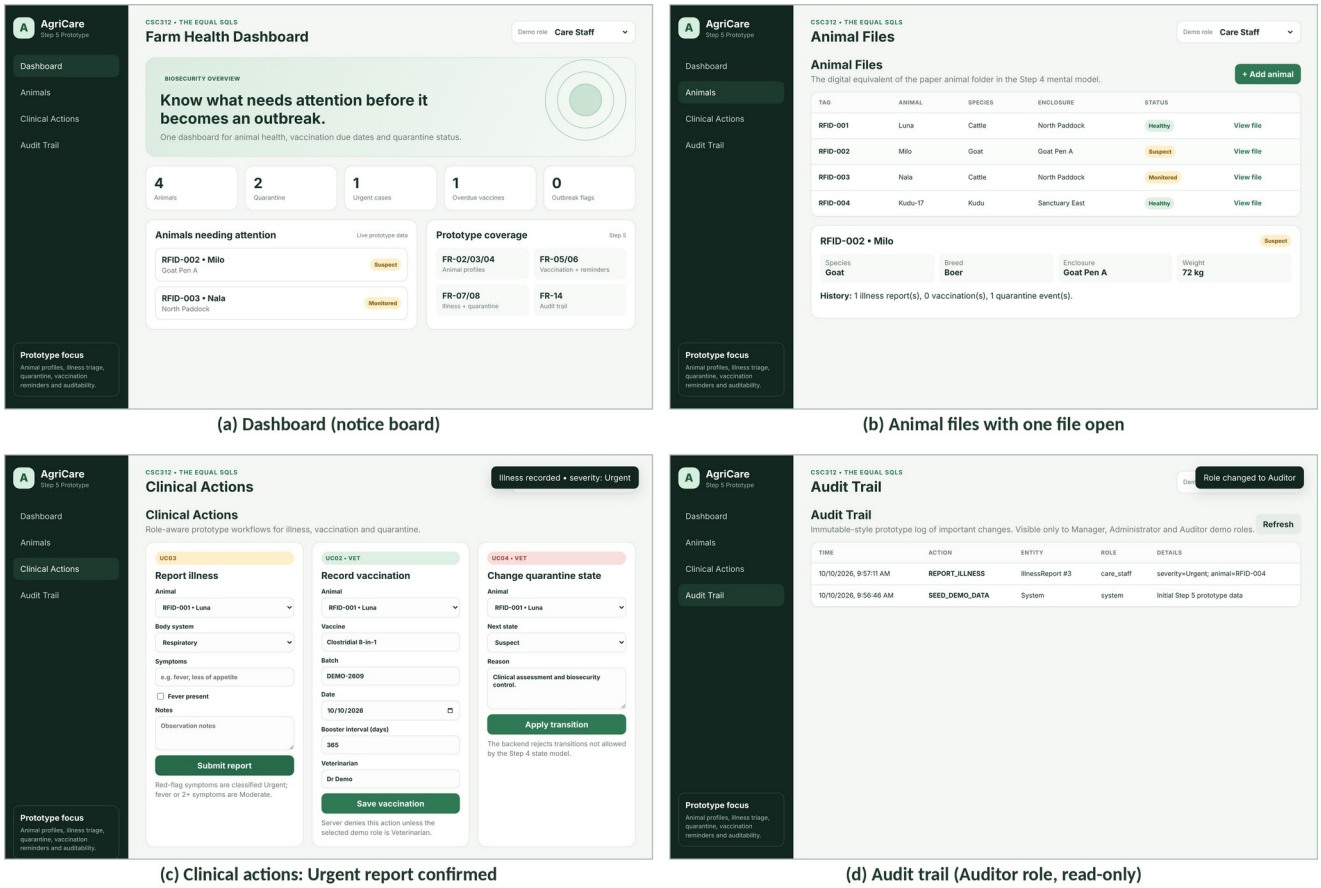


Figure 9: Screens of the program-code prototype, captured from the running application with the seeded demonstration data.

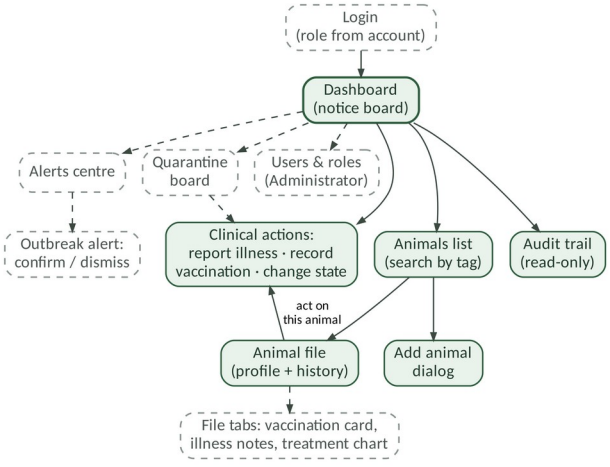


Figure 8: Navigation map. Solid screens exist in the prototype; dashed screens are planned for the next iteration.

Four general design issues were also addressed. *Response time*: the dashboard needs only two small API calls. *Help and error handling*: plain-language messages, such as "Only the Veterinarian role can record vaccinations". *Accessibility*: every input has a visible label, status colour is always paired with a word, and native form controls work from the keyboard [10]. *Responsiveness*: at phone width (390 px) the sidebar becomes a top bar and the dashboard cards stack, which was checked in a mobile viewport. isiZulu support (FR-17) is planned by keeping all interface text in one place so it can be translated.

5.6.4 Design Evaluation. Reviewing the prototype against the golden rules and the mental model produced four changes for the next iteration:

- **Symptom checklist instead of free text.** Symptoms are typed as a comma-separated list and matched exactly against the red-flag list. In testing, "Breathing difficulties" (plural) was classed *Low* instead of *Urgent*. A checklist of coded symptoms grouped by body system removes this safety risk and is also faster to use with gloves.

- **Offer only permitted next states.** The quarantine form lists all five states, although only one or two are valid at a time. The server rejects invalid choices, but the form should show only the permitted gates, matching the "sick bay and gates" metaphor and preventing the error.
- **Login instead of a role selector.** The demonstration role box is replaced by login, with the role read from the user's account.
- **Tabs in the animal file.** The file currently shows only counts of history items. Tabs for the vaccination card, illness notes, quarantine timeline and treatment chart will complete the "animal file" anchor.

A usability test is planned once these changes are in. Five care-staff participants will each record an illness report (target: 60 seconds or less, NFR-06), then complete the System Usability Scale questionnaire.

6 PROTOTYPE DEVELOPMENT

6.1 Purpose and Scope

Step 5 converts the analysis and design models into executable program code. The prototype is an *evolutionary* prototype and intentionally a vertical slice: instead of attempting every requirement at once, it implements a small set of high-value workflows end-to-end so that the code can be extended into the completed AgriCare application. The implemented slice is traced directly to the use cases and functional requirements: animal profiles (UC01), vaccinations and booster due dates (UC02), illness reporting and severity classification (UC03), quarantine transitions (UC04), a shared dashboard (UC08; role-specific views are planned), outbreak flagging as an early version of UC06, and audit logging (UC10).

The prototype keeps the domain rules established during requirements analysis. For example, a red-flag symptom is classified as Urgent, Moderate or Urgent reports can place a healthy animal into Suspect status, quarantine transitions are constrained by the state model, and only the Veterinarian demo role can record vaccinations or change quarantine state. These rules are implemented on the server rather than only in the interface, so changing the user interface later does not remove the validation.

6.2 Prototype Architecture

The prototype implements the layered architecture of Section 5.2 (Figure 5). The browser interface communicates with a FastAPI REST layer [16] using JSON. The API validates input and role permissions before calling domain rules, and SQLAlchemy [17] isolates persistence from the API. For demonstration the default database is SQLite, because it can run on any group member's laptop without database-server configuration; the same code can target MySQL by setting `DATABASE_URL`. MySQL remains the database selected for the completed system.

6.3 Hardware and Software Environment

No specialist hardware is required. The recommended demonstration environment is a normal student or laboratory computer with an x64 processor, 4 GB of RAM or more, about 500 MB of free storage and a modern browser. Internet access is not required for the core local demo; it will become necessary when the weather API and hosted deployment are added. The interface is responsive, so the same prototype can be demonstrated on a laptop and tested at mobile widths. Table 12 summarises the toolchain.

Table 12: Development environment and toolchain

Area	Prototype choice and purpose
Backend	Python 3 with FastAPI for REST endpoints, validation and business rules; Uvicorn as the server.
Persistence	SQLAlchemy 2 ORM; SQLite for the portable prototype and MySQL as the target database.
Frontend	HTML, CSS and JavaScript, separated from the API so it can be replaced by the planned React interface without changing backend contracts.
Testing	pytest for automated rule tests; Postman or command-line HTTP requests for endpoint checks.
Database tools	MySQL Workbench when the MySQL configuration is enabled.
Version control	Git/GitHub for source control, review and change history.
Design and docs	UML models, Figma/Draw.io artefacts and this ACM-format report guide implementation.

6.4 Implemented Key Functionality

Table 13 records what has moved from requirement to executable code. The purpose is not to claim that the prototype is the completed product; it establishes a working base around which later sprints can add treatment records, weather integration, notifications, complete authentication, security incidents and the remaining nice-to-have requirements. Figure 9 shows the main screens.

6.5 Prototype Data and Behaviour

The data model in code is a deliberately smaller implementation of the data design in Section 5.3 (the dark tables in Figure 6). The prototype persists Animal, IllnessReport, VaccinationRecord, QuarantineEvent, OutbreakAlert and AuditLog. Foreign-key relationships link clinical events to the animal file, so the application can show one animal's history while keeping each event record independently auditable.

Table 13: Traceability from requirements to prototype evidence

Requirement	Prototype function	Executable behaviour	Evidence in code / UI
UC01; FR-02–04	Animal profiles	Create, list and open active animal files identified by a unique tag ID; duplicate tag IDs are rejected.	/api/animals; Animals screen
UC02; FR-05–06	Vaccination	Veterinarian records vaccine data; the next booster due date is calculated from administration date plus interval.	/vaccinations; Clinical Actions screen
UC03; FR-07	Illness report	Symptoms are saved and classified Low, Moderate or Urgent; red-flag symptoms become Urgent.	/illness; automated rule tests
UC04; FR-08	Quarantine	Veterinarian-only transitions are checked against Healthy → Suspect → Isolated → Monitored → Cleared, with permitted backward transitions only.	/quarantine; transition table in code
UC06; FR-10	Outbreak flag	Counts distinct animals with reports in the same enclosure and body system within 72 hours; three or more creates an alert.	OutbreakAlert; dashboard count
UC08; FR-12	Dashboard	Shows total animals, animals in quarantine, urgent reports, overdue vaccination records and open outbreak flags.	Dashboard page; /api/dashboard
UC10; FR-14	Audit trail	Create, report, vaccination and quarantine actions write an audit record; access to the log is role-limited.	AuditLog; Audit Trail screen
NFR-02/04/09	Security, integrity, maintainability	Server-side role checks, duplicate and transition validation, and separate database, model, schema and API modules.	Backend modules; 403/400 responses

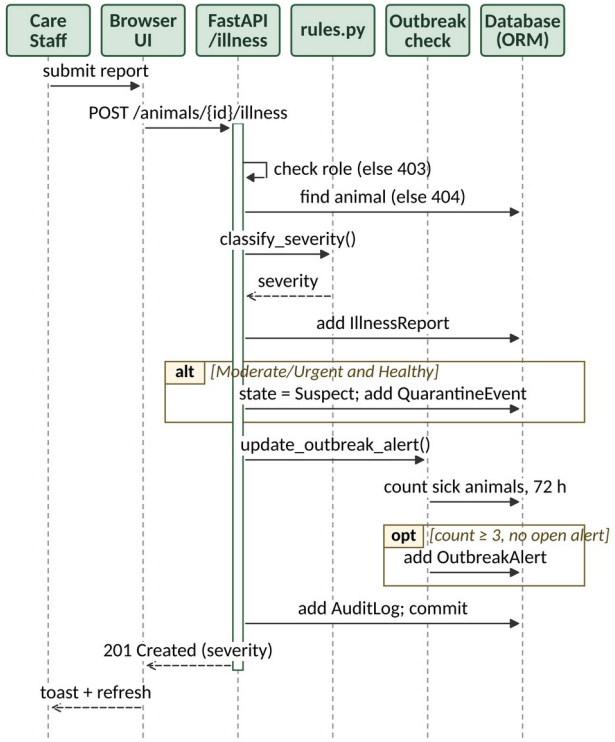


Figure 10: UML sequence diagram of illness reporting (UC03) as implemented in the prototype, from the form submission to the audit entry.

Several behavioural rules from Section 4.5 are now executable, following Listings 1–3; Figure 10 shows how the components cooperate when an illness is reported. The severity function treats breathing difficulty, collapse, mouth lesions, hoof lesions or sudden death as red flags. Fever or two or more non-red-flag symptoms produces Moderate severity; otherwise the result is Low. A Moderate or Urgent report on a Healthy animal creates a Suspect transition. Vaccination due dates are calculated rather than typed by hand. The quarantine API checks each requested state change against the permitted transition set and rejects an invalid jump such as Healthy directly to Cleared.

6.6 Verification of the Prototype

Testing is part of the prototype rather than an activity postponed until the final product. All five automated unit tests pass; they exercise severity classification, quarantine transition rules and booster-date calculation. Manual API checks were also performed on the main access-control path: a Care Staff request to record a vaccination receives HTTP 403, while the same valid request under the Veterinarian role is accepted and the next due date is calculated. An Urgent illness report was verified to return Urgent severity and place the animal into Suspect status (Table 14).

Table 14: Verification evidence for the prototype

Check	Expected result	Result
Rule tests	5 pytest tests pass	Pass
Red flag	“breathing difficulty” → Urgent	Pass
Access control	Care Staff vaccination request rejected	403 (pass)
Vaccination	365-day interval computes next due date	Pass
Transition	Healthy → Cleared rejected	400 (pass)
Exploratory: symptom wording	“Breathing difficulties” → Urgent	Low (defect; Section 5.6.4)

The last row is a separate exploratory check, not one of the five automated tests. It was added during design evaluation and exposed a defect: severity matching depends on exact wording. The defect is kept on record because finding it is exactly what a prototype is for, and the symptom checklist in Section 5.6.4 is the planned fix. Performance targets such as the 300 ms requirement have not yet been formally load-tested, so no compliance with those targets is claimed. They remain acceptance criteria for a later sprint, once the MySQL and deployment environments are stable.

6.7 Prototype Access

The source code, automated tests, documentation and this report are in the group's public GitHub repository: **https://github.com/callmetumz/AgriCare**. The prototype runs locally on Windows, macOS or Linux with Python 3 installed. Install the dependencies with `pip install -r requirements.txt`, then start it with `run_demo.bat` (Windows) or `run_demo.sh` (macOS/Linux), which seed the demonstration data and start the server. Then open http://127.0.0.1:8000 in a browser and choose a demo role at the top right. The automated tests run with `python -m pytest`.

7 CONTINUED APPLICATION OF SOFTWARE ENGINEERING

7.1 Process

The group continues to use Agile/Scrum [14]. The use cases and requirements act as the product backlog; Step 5 selected a small vertical slice of the must-have backlog for implementation. The practical sprint cycle is: select backlog items, refine acceptance criteria, implement the smallest working path, test it, review the result against the requirement, and return incomplete or newly discovered work to the backlog. The prototype is therefore an evolutionary base rather than disposable code.

Each use case becomes one or more user stories whose acceptance criteria are written as *Given / When / Then* statements before coding. For example, *given* an animal that has been Monitored for 5 days of a 14-day period, *when* the vet selects Cleared, *then* the system rejects the change and shows when discharge becomes possible. These criteria become the automated tests required by the test-driven approach.

7.2 Principles

Separation of concerns is visible in the code structure: database setup, ORM models, validation schemas, domain rules, HTTP endpoints and browser assets are separated. This reduces coupling and makes the planned React migration possible without rewriting domain rules. Least privilege is applied by checking the role at server endpoints (Table 9); for example, only the Veterinarian demo role may record vaccinations or change quarantine state, while audit viewing is limited to audit-relevant roles. Defence in depth is only partly implemented at prototype level: server-side checks exist even if an interface control is hidden, but production authentication, HTTPS, secure credential storage and full authorisation remain future work. Data integrity is supported through required fields, unique animal tags, foreign keys and rejected invalid state transitions.

7.3 Methods and Techniques

The prototype continues the selected methods of UML-guided development and test-driven development. The use-case, class and state models are translated into endpoints and executable transition rules, preserving traceability from analysis through design to code. Rule tests are written around behaviours that are easy to break, such as severity and quarantine progression. API-based prototyping also gives the group a stable interface contract: the browser client can later be replaced with React while calling the same endpoints.

7.4 Software Quality Goals

The product-quality characteristics of ISO/IEC 25010 [20] guide implementation. *Functional suitability* is addressed by tracing each implemented function to a requirement (Table 13). *Usability* is addressed through short forms, a dashboard-first layout and plain-language status labels. *Accessibility* is supported by pairing words with status colour and using responsive controls. *Security* is represented by server-side role checks, while the current header-based role simulation is explicitly not treated as production authentication. *Reliability* and integrity are supported by input validation and fail-fast error responses. *Maintainability* is supported by modular source files and automated tests. *Compatibility* is supported through a browser-based interface and JSON API, with the database configuration kept outside the code in an environment variable.

7.5 Umbrella Activities

Configuration management continues through version control and a source tree that every group member can review. *Technical reviews* compare each implementation with its originating requirement and use-case flow; every use case, diagram and requirement is peer-reviewed against a checklist (unique ID, testable wording, actor present, traced to a stakeholder). *Risk management* is updated as implementation reveals new risks: the most important current risks are prototype authentication being mistaken for final security, differences between SQLite and MySQL, false positives from the simple outbreak rule, free-text symptom matching, and schedule pressure from unimplemented weather and treatment functionality. *Documentation* is kept with the code in a README and a requirement-traceability file. *Measurement* currently covers test results and implemented-requirement coverage; later sprints will add performance measurements and defect counts.

7.6 Tools

FastAPI and Python implement the service layer; SQLAlchemy provides database independence; SQLite provides a zero-configuration demonstration database, while MySQL remains the final target. HTML, CSS and JavaScript implement the current interface, with React planned as the evolved front end. pytest

supports automated tests, Postman or command-line requests support API verification, MySQL Workbench supports the final database, Git/GitHub supports source control and collaboration, Figma supports interface refinement, Draw.io supports UML and architecture updates, and Word and PDF tooling supports the ACM-format documentation.

8 PROTOTYPE LIMITATIONS AND EVOLUTION PLAN

The current prototype deliberately does not represent the completed AgriCare system. The role selector sends a demonstration role header instead of performing secure authentication; this must be replaced with registered users, password hashing and server-managed identity. Weather integration, treatment records, security-incident workflows, notification delivery, full archive and update operations, discharge checks (BR-06), richer outbreak confirmation and the remaining nice-to-have requirements are not yet complete. Design evaluation also found four gaps between the prototype and the design: symptoms are matched as free text, the quarantine form offers states that are not permitted, the dashboard shows the same counts to every role rather than role-specific views, and the overdue count includes doses that a newer dose has already replaced (Listing 3).

The next sprint should therefore (1) implement secure login and user and role persistence, (2) replace free-text symptoms with the coded checklist and show only the permitted next quarantine states, (3) migrate and test the schema on MySQL, (4) add treatment history and complete animal editing and archiving, (5) integrate a weather provider behind the `WeatherService` interface with cached fallback data, (6) convert the browser views into React components, and (7) run usability, accessibility and performance tests against the measurable NFR targets. This keeps the Step 5 code as the foundation of the completed application rather than replacing it with a new implementation.

9 CONCLUSIONS

This report modelled AgriCare from its users' point of view and then designed and prototyped it. The user model profiled six human roles and one external system and mapped their tasks to eleven use cases. The mental model presents AgriCare as a digital farm office with a sick bay, built on four familiar anchors. The requirements analysis modelled the system's data, function and behaviour and set out eighteen functional and nine non-functional requirements. The design model specifies a layered REST architecture, a relational schema, typed design classes, pseudocode for the safety-critical components and an interface design guided by the golden rules. The program-code prototype implements a traced vertical slice of that design, passes its automated tests, and has already exposed design improvements, notably the symptom checklist. Together these form the basis for the implementation sprints that follow.

REFERENCES

[1] The Equal SQLs. 2026. *AgriCare: Bridging the Gap Between Software and Farmers*. Project Proposal, CSC312 Software Engineering, University of the Western Cape.

[2] World Organisation for Animal Health (WOAH). 2024. *Terrestrial Animal Health Code*. WOAH, Paris.

[3] Ben Shneiderman, Catherine Plaisant, Maxine Cohen, Steven Jacobs, Niklas Elmqvist, and Nicholas Diakopoulos. 2016. *Designing the User Interface: Strategies for Effective Human-Computer Interaction* (6th ed.). Pearson, Boston, MA.

[4] Alistair Cockburn. 2000. *Writing Effective Use Cases*. Addison-Wesley, Boston, MA.

[5] Martin Fowler. 2003. *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley, Boston, MA.

[6] Donald A. Norman. 2013. *The Design of Everyday Things* (revised and expanded ed.). Basic Books, New York, NY.

[7] CSC312 course. 2026. *Step 4: Requirements and Analysis*. Assignment instructions supplied to the group, University of the Western Cape.

[8] Alan Cooper, Robert Reimann, David Cronin, and Christopher Noessel. 2014. *About Face: The Essentials of Interaction Design* (4th ed.). Wiley, Indianapolis, IN.

[9] Jakob Nielsen. 1994. Enhancing the explanatory power of usability heuristics. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (CHI '94). ACM, New York, NY, 152–158.

[10] W3C. 2023. *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation. https://www.w3.org/TR/WCAG22/

[11] Ian Sommerville. 2016. *Software Engineering* (10th ed.). Pearson, Boston, MA.

[12] Roger S. Pressman and Bruce R. Maxim. 2015. *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill Education, New York, NY.

[13] Republic of South Africa. 2013. *Protection of Personal Information Act, No. 4 of 2013*. Government Gazette.

[14] Ken Schwaber and Jeff Sutherland. 2020. *The Scrum Guide*. https://scrumguides.org

[15] Martin Fowler. 2002. *Patterns of Enterprise Application Architecture*. Addison-Wesley, Boston, MA.

[16] Sebastián Ramírez. 2026. *FastAPI Documentation*. https://fastapi.tiangolo.com

[17] SQLAlchemy Authors. 2026. *SQLAlchemy 2.0 Documentation*. https://docs.sqlalchemy.org

[18] Peter Pin-Shan Chen. 1976. The entity-relationship model: toward a unified view of data. *ACM Trans. Database Syst.* 1, 1 (March 1976), 9–36.

[19] Theo Mandel. 1997. *The Elements of User Interface Design*. John Wiley & Sons, New York, NY.

[20] ISO/IEC. 2023. *ISO/IEC 25010:2023 Systems and Software Engineering: Systems and Software Quality Requirements and Evaluation (SQuaRE): Product Quality Model*. International Organization for Standardization, Geneva.